

Experiment ‘p_6_2_h_is_sorted_array’ Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_6_2_h_is_sorted_array

Natural language statement: Write a method to return true if the given array is currently sorted in increasing order.

Method signature: p_6_2_h_is_sorted_array(inputs: array<int>) returns (is_sorted: bool)

Ensures

- `is_sorted == (forall i :: 0 <= i < inputs.Length - 1 ==> inputs[i] <= inputs[i + 1])`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to return true if the given array is currently sorted in increasing order.

The signature should be:

`p_6_2_h_is_sorted_array(inputs: array<int>) returns (is_sorted: bool)`

The method should respect the following contract:

`ensures is_sorted == (forall i :: 0 <= i < inputs.Length - 1 ==> inputs[i] <= inputs[i + 1])`

Produce and show only the Dafny body of this method, including the curly braces that surround it. Do not show the signature nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn1ElBGvkxY00owmxEgnhF9tEwwWU

Model: gpt-5.5-2026-04-23

Created at: 1780575199

Finish reason: stop

Usage: 312 tokens in, and 8481 tokens out

```
{
  is_sorted := true;
  var n := inputs.Length;
  var i := 0;

  while i < n - 1
    invariant n == inputs.Length
    invariant 0 <= n
    invariant 0 <= i
    invariant n == 0 ==> i == 0
    invariant n > 0 ==> i <= n - 1
    invariant is_sorted ==> (forall j :: 0 <= j < i ==> inputs[j] <= inputs[j + 1])
    invariant !is_sorted ==> (exists j :: 0 <= j < i && inputs[j] > inputs[j + 1])
    decreases n - i
  {
    assert n > 0;
    assert i + 1 < n;

    if inputs[i] > inputs[i + 1] {
      is_sorted := false;
    }
  }
}
```

```

    assert 0 <= i < i + 1 && inputs[i] > inputs[i + 1];
    assert (exists j :: 0 <= j < i + 1 && inputs[j] > inputs[j + 1]);
  } else {
    assert inputs[i] <= inputs[i + 1];
    if is_sorted {
      assert (forall j :: 0 <= j < i + 1 ==> inputs[j] <= inputs[j + 1]) by {
        forall j | 0 <= j < i + 1
          ensures inputs[j] <= inputs[j + 1]
        {
          if j < i {
            assert inputs[j] <= inputs[j + 1];
          } else {
            assert j == i;
            assert inputs[i] <= inputs[i + 1];
          }
        }
      }
    } else {
      assert (exists j :: 0 <= j < i && inputs[j] > inputs[j + 1]);
      var j :| 0 <= j < i && inputs[j] > inputs[j + 1];
      assert 0 <= j < i + 1 && inputs[j] > inputs[j + 1];
      assert (exists k :: 0 <= k < i + 1 && inputs[k] > inputs[k + 1]);
    }
  }

  i := i + 1;
}

assert !(i < n - 1);
if n > 0 {
  assert i == n - 1;
}

if is_sorted {
  assert (forall k :: 0 <= k < n - 1 ==> inputs[k] <= inputs[k + 1]) by {
    forall k | 0 <= k < n - 1
      ensures inputs[k] <= inputs[k + 1]
    {
      if n == 0 {
        assert false;
      } else {
        assert i == n - 1;
        assert k < i;
        assert inputs[k] <= inputs[k + 1];
      }
    }
  }
} else {
  assert (exists j :: 0 <= j < i && inputs[j] > inputs[j + 1]);
  var j :| 0 <= j < i && inputs[j] > inputs[j + 1];
  if n == 0 {

```

```

    assert i == 0;
    assert false;
  } else {
    assert i == n - 1;
  }
  assert 0 <= j < n - 1 && inputs[j] > inputs[j + 1];
  assert !(inputs[j] <= inputs[j + 1]);
  if (forall k :: 0 <= k < n - 1 ==> inputs[k] <= inputs[k + 1]) {
    assert inputs[j] <= inputs[j + 1];
    assert false;
  }
  assert !(forall k :: 0 <= k < n - 1 ==> inputs[k] <= inputs[k + 1]);
}

assert n == inputs.Length;
assert is_sorted == (forall k :: 0 <= k < inputs.Length - 1 ==> inputs[k] <= inputs[k +
  ↪ 1]);
}

```

Final Program

```

method p_6_2_h_is_sorted_array(inputs: array<int>) returns (is_sorted: bool)
  ensures is_sorted == (forall i :: 0 <= i < inputs.Length - 1 ==> inputs[i] <=
    ↪ inputs[i + 1])
{
  is_sorted := true;
  var n := inputs.Length;
  var i := 0;

  while i < n - 1
    invariant n == inputs.Length
    invariant 0 <= n
    invariant 0 <= i
    invariant n == 0 ==> i == 0
    invariant n > 0 ==> i <= n - 1
    invariant is_sorted ==> (forall j :: 0 <= j < i ==> inputs[j] <= inputs[j + 1])
    invariant !is_sorted ==> (exists j :: 0 <= j < i && inputs[j] > inputs[j + 1])
    decreases n - i
  {
    assert n > 0;
    assert i + 1 < n;

    if inputs[i] > inputs[i + 1] {
      is_sorted := false;
      assert 0 <= i < i + 1 && inputs[i] > inputs[i + 1];
      assert (exists j :: 0 <= j < i + 1 && inputs[j] > inputs[j + 1]);
    } else {
      assert inputs[i] <= inputs[i + 1];
      if is_sorted {
        assert (forall j :: 0 <= j < i + 1 ==> inputs[j] <= inputs[j + 1]) by {

```

```

    forall j | 0 <= j < i + 1
      ensures inputs[j] <= inputs[j + 1]
    {
      if j < i {
        assert inputs[j] <= inputs[j + 1];
      } else {
        assert j == i;
        assert inputs[i] <= inputs[i + 1];
      }
    }
  }
} else {
  assert (exists j :: 0 <= j < i && inputs[j] > inputs[j + 1]);
  var j :| 0 <= j < i && inputs[j] > inputs[j + 1];
  assert 0 <= j < i + 1 && inputs[j] > inputs[j + 1];
  assert (exists k :: 0 <= k < i + 1 && inputs[k] > inputs[k + 1]);
}
}

i := i + 1;
}

assert !(i < n - 1);
if n > 0 {
  assert i == n - 1;
}

if is_sorted {
  assert (forall k :: 0 <= k < n - 1 ==> inputs[k] <= inputs[k + 1]) by {
    forall k | 0 <= k < n - 1
      ensures inputs[k] <= inputs[k + 1]
    {
      if n == 0 {
        assert false;
      } else {
        assert i == n - 1;
        assert k < i;
        assert inputs[k] <= inputs[k + 1];
      }
    }
  }
} else {
  assert (exists j :: 0 <= j < i && inputs[j] > inputs[j + 1]);
  var j :| 0 <= j < i && inputs[j] > inputs[j + 1];
  if n == 0 {
    assert i == 0;
    assert false;
  } else {
    assert i == n - 1;
  }
  assert 0 <= j < n - 1 && inputs[j] > inputs[j + 1];
}

```

```
    assert !(inputs[j] <= inputs[j + 1]);
    if (forall k :: 0 <= k < n - 1 ==> inputs[k] <= inputs[k + 1]) {
        assert inputs[j] <= inputs[j + 1];
        assert false;
    }
    assert !(forall k :: 0 <= k < n - 1 ==> inputs[k] <= inputs[k + 1]);
}

assert n == inputs.Length;
assert is_sorted == (forall k :: 0 <= k < inputs.Length - 1 ==> inputs[k] <= inputs[k +
    ↪ 1]);
}
```

Total Token Usage

Input tokens: 312

Output tokens: 8481

Reasoning tokens: 7459

Sum of 'total tokens': 8793

Experiment Timings

Overall Experiment started at 1780575198830, ended at 1780575399983, lasting 201153ms (201.15 seconds)

Iteration #1 started at 1780575198830, ended at 1780575399983, lasting 201153ms (201.15 seconds)